A one-body context calculus for intuitionistic primitive recursive arithmetic

Exact equivalence, complete rules, and measured implementation costs

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The raw clause presentation of ConciseIPRA retains pairs $[A; y.B(y)]$, with ordinary reading $A \to \forall y B(y)$. We remove the independent antecedent field: every retained entry is just a body $B(y)$, read $B(0) \to \forall y B(y)$. Ordinary assumptions are represented by a pointwise iterator gate. The resulting calculus uses the same raw terms, the same equation/D formulas, atomic reflexivity, the existing two-case evaluation rule, and three logical rules. We prove that it has exactly the same open consequences as the established native calculus, under an explicit representation of assumptions. Consequently it is bi-interpretable and equiconsistent with full first-order iPRA, with whole numerical domains, actual equality, and both numerical composites given by $x = y$.

The size result is deliberately qualified. The complete new checker is 3,130 normalized bytes, versus 2,999 for a matched two-field control. Thus the context grammar is simpler, but this implementation does not improve the repository's proof-checker-size criterion. A separate, semantics-preserving implementation refinement of the existing full Eq/D checker reduces 788 bytes to 778, while also reducing its token and AST counts; a 777-byte variant also included adds two AST nodes to the baseline. These refinements introduce no mathematical change. This package therefore does *not* claim a new globally smallest axiomatization or a mathematical size breakthrough.

1 Result and comparison boundary

The comparison snapshot is ProveIt commit

`11814771b7490dfbea0ed5fcf0f5b23586334764`.

The inspected directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README describes seventy-three research packages, multiple concise calculi, and the Lean bridge from the native scalar language to iPRA [1, 6]. Several familiar reductions are already present: one-term transport, equality from iteration, clause contexts, zero and full normalization, and the fusion of equality with witness discharge. Those are not claimed as discoveries of this report.

The new observation concerns the *origin of contexts*. In a raw clause proof starting from ordinary assumptions, a retained clause arises in only two ways:

$$\mathrm{Asm}(A) = [0 = 0; v.A], \qquad \mathrm{Self}(B) = [B(0); y.B(y)].$$

The first represents an assumption; the second comes from opening a paired formula. We show that the first can also be represented by an entry of the second kind. The primitive context datatype can therefore lose its independent antecedent field entirely.

Theorem 1.1 (Main equivalence, stated informally)**.** *For Eq/D formulas $A_1, \dots, A_n, F$, the new calculus derives F from the retained bodies $\mathsf{G}A_1, \dots, \mathsf{G}A_n$ if and only if the established native calculus derives F from ordinary assumptions $A_1, \dots, A_n$.*

The theorem is about arbitrary finite proofs and open consequences. It is stronger than agreement on equations in the standard natural numbers. Its proof does not require a normalizer, a predecessor construction, a new cut theorem, or unrestricted substitution into witness-selected logical derivations.

What remains unmet. Under the earlier working criterion—the size of a complete standalone proof checker—the new calculus has not beaten the current proposals. The positive mathematical result is the deletion of one context field and the corresponding exact equivalence theorem. The separate 10-byte implementation improvement concerns the existing calculus. Combining these into one purported new record would be incorrect.

2 Complete presentation on raw syntax

2.1 Terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in the last child h is bound by I. Terms are *raw*: $I(0, a; u.h)$ and a are different syntax trees. All instances below use capture-avoiding substitution; syntactic equality is up to consistent renaming of bound variables, or literal equality in the delivered de Bruijn representation.

The complete formula grammar is

$$A ::= (s = t) \mid D_{x,y}A.$$

Both x and y are bound in the matrix of D. There are no primitive propositional connectives, quantifiers other than D, or separate Q formulas. A formula is a finite D prefix ending in a single equation. The designated contradiction is $0 = S0$.

The *ordinary reading* of the native binder is

$$\big(D_{x,y}B\big)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big). \tag{D}$$

This is an interpretation into ordinary intuitionistic first-order logic, not an additional primitive inference rule.

2.2 One-body contexts and the gate

A context entry is a single formula body $B(v)$ with one distinguished bound numerical coordinate v. Its meaning is fixed:

$$\mathcal{Q}_v B := B^\circ(0) \to \forall v\, B^\circ(v). \tag{Context}$$

A context is a finite list of such bodies. The binder belonging to an entry is distinct from all D binders inside that entry. There is no independent antecedent field and no primitive ordinary-assumption entry.

Put

$$\mathsf{C}(v, a, b) := I(v, a; u.b), \qquad u \notin \mathrm{FV}(b).$$

For $A = D_{x_1,y_1} \cdots D_{x_d,y_d}(s = t)$ and a fresh coordinate v, define the abbreviation

$$\mathsf{G}_v A := D_{x_1,y_1} \cdots D_{x_d,y_d}(\mathsf{C}(v, 0, s) = \mathsf{C}(v, 0, t)). \tag{Gate}$$

The fresh iterator variables are unused. Every old bound coordinate keeps its binding. In particular, v is placed outside all $2d$ logical coordinates, while the endpoint terms are lifted once when placed under the new iterator binder. Gate is a construction on the existing grammar, not a new formula constructor. For a formula family $A(v)$, the same pointwise construction defines $\mathsf{G}_v A(v)$; the coordinate must stay fresh for its internal binders. In its use to represent an ordinary assumption, v is additionally absent from A.

2.3 All primitive rules

Write $\Gamma \vdash_{\mathsf{B}} A$ for the least derivability relation generated by the following five rule families. Eval has two cases, so there are six displayed inference schemata in total.

R: atomic reflexivity. For any well-scoped numerical term t,

$$\frac{}{\Gamma \vdash_{\mathsf{B}} t = t}. \tag{R}$$

Reflexivity under a D prefix is derived later; it is not part of R.

Eval: computational equality.

$$\frac{\Gamma \vdash_{\mathsf{B}} a = t \qquad \Gamma \vdash_{\mathsf{B}} I(0, a; u.h) = r}{\Gamma \vdash_{\mathsf{B}} r = t}, \tag{E_0}$$

$$\frac{\Gamma \vdash_{\mathsf{B}} I(c, a; u.h) = t \qquad \Gamma \vdash_{\mathsf{B}} I(Sc, a; u.h) = r}{\Gamma \vdash_{\mathsf{B}} r = h[t/u]}. \tag{E_S}$$

There is no independent root-joining, congruence, symmetry, transitivity, or computation axiom. Both premises of Eval are literal equations.

Use. If $B(v)$ is a retained context occurrence and t is a term,

$$\frac{\Gamma \vdash_{\mathsf{B}} B(0)}{\Gamma \vdash_{\mathsf{B}} B(t)}. \tag{Use}$$

Open.

$$\frac{\Gamma \vdash_{\mathsf{B}} D_{x,y} B(x, y) \qquad \Gamma, B(b, y) \vdash_{\mathsf{B}} A}{\Gamma \vdash_{\mathsf{B}} A}. \tag{Open}$$

The second premise adds the single body $B(b, y)$. The eigenparameter b is fresh for Γ, A, and all passive parameters of B.

Intro.

$$\frac{\Gamma, \mathsf{G}_v(B(w,\eta)) \vdash_{\mathsf{B}} B(w, Sz)}{\Gamma \vdash_{\mathsf{B}} D_{x,y}B(x,y)}, \tag{Intro}$$

where z is fresh for Γ, w, and the passive parameters of B, and

$$\eta = \begin{cases} z, & \text{if } B \text{ is a literal equation and } w \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Mode}$$

The distinguished gate coordinate v is fresh and separate from z. The mode test is on syntax, before any equation reasoning. It is the existing witness-selected introduction, with its temporary assumption represented by a gate body.

These rules and the finite syntax are the entire presentation. In particular, ordinary intuitionistic logic, numerical case distinction, equation induction, weakening, ordinary hypothesis access, and cut are not additional primitive rules.

3 Equality is recovered before the logical argument

The following calculations use only R, E_0, and E_S. They work in every retained context and are the arithmetic interface already used by the repository's two-case Eval calculus [3].

Lemma 3.1 (Computation and joining)**.** *Both iterator computation equations are derivable. Moreover, from $a = b$ and $a = c$ one can derive $c = b$.*

Proof. For zero computation, apply E_0 to $a = a$ and $I(0, a; u.h) = I(0, a; u.h)$. For successor computation, apply E_S to the reflexivities of the preceding and current iterates.

For joining put $W = I(0, a; u.u)$. Apply E_0 to $a = c$ and $W = W$ to obtain $W = c$. Apply E_0 again to $a = b$ and $W = c$ to obtain $c = b$. □

Lemma 3.2 (Crossed one-term transport)**.** *From $a = b$ and $h(a) = r$ one can derive $r = h(b)$.*

Proof. Joining and R first give symmetry and transitivity. Put

$$W = I(0, a; u.h(u)), \qquad J = I(S0, a; u.h(u)).$$

Zero Eval gives $W = a$. Successor Eval applied directly to $W = a$ and $J = J$ gives $J = h(a)$. This step does not use congruence. Joining and transitivity give $W = b$ and $J = r$ from the two supplied equalities. Successor Eval applied to these gives $r = h(b)$. □

Consequently the pure subsystem provides ordinary equational reasoning in every term template, including repeated occurrences and nested iterator binders. Write $s \equiv_E t$ if this subsystem proves $s = t$, with the displayed numerical parameters free. These pure derivations are stable under capture-avoiding numerical substitution: their rules have no witness-dependent mode test.

In particular,

$$\mathsf{C}(0, 0, s) \equiv_E 0, \qquad \mathsf{C}(Sz, 0, s) \equiv_E s. \tag{Endpoints}$$

The second identity holds because the update ignores its iterator coordinate. It does not require evaluating the preceding iterate.

4 Gated assumptions are accessible without circularity

Lemma 4.1 (Derived deep reflexivity)**.** *Every well-scoped D prefix above an equation $t = t$ is derivable in the new calculus, in any context.*

Proof. Induct externally on the number of D nodes. At depth zero use R. At a node $D_{x,y}B$, apply Intro with witness $1 = S0$. This selects case mode. The successor instance of B again ends in a reflexive equation, so the induction hypothesis proves it in the extended context, ignoring the new gate body. No access to that body is needed. □

Weakening and injective renaming of free parameters are admissible by direct induction on derivations, after freshening eigenparameters. Injective renaming preserves the distinction between literal zero and a variable. We do not assume that arbitrary substitution preserves an Intro proof tree: it can change a nonzero witness into literal zero.

For formulas with the same alpha-aligned D prefix, define $A \simeq_E A'$ when their corresponding left endpoints, and their corresponding right endpoints, are related by $\equiv_E$. Bound coordinates in these endpoint equations are temporarily treated as free numerical parameters. This definition does *not* presuppose formula transport.

For every depth d we establish two assertions in this order:

L_d**.** If $A \simeq_E A'$ and both have depth d, a derivation of either formula in any retained-body context can be converted into a derivation of the other.

H_d**.** If A has depth d and $\mathsf{G}_v A$ is a retained body, then $\Gamma \vdash_{\mathsf{B}} A$.

Lemma 4.2 (Conversion and access)**.** *All assertions L_d and H_d hold.*

Proof. At depth zero L_0 is equational congruence from Section 3.

Suppose L_{d-1} and H_{d-1} have been established. Let $A = D_{x,y}B$ and $A' = D_{x,y}B'$. Open a supplied proof of A at fresh b, retaining the body $B(b, y)$. Introduce A' with witness b. A fresh variable is not literally zero, so this is case mode, even when the matrix is an equation. The inner context now contains $\mathsf{G}_v B'(b, 0)$.

Use H_{d-1} to obtain $B'(b, 0)$. Apply L_{d-1}, instantiated at $x = b, y = 0$, to obtain $B(b, 0)$. Use the retained body $B(b, y)$ at Sz to obtain $B(b, Sz)$. Apply L_{d-1} to obtain $B'(b, Sz)$. Intro proves A', and Open discharges b. This proves L_d in one direction; the other uses the symmetric endpoint equations.

Having established L_d, let T_d denote the same D prefix above $0 = 0$. Derived deep reflexivity proves T_d. The endpoint equations give

$$\mathsf{G}_0 A \simeq_E T_d, \qquad \mathsf{G}_1 A \simeq_E A.$$

Apply L_d to get $\mathsf{G}_0 A$. Use the retained body $\mathsf{G}_v A$ at 1, obtaining $\mathsf{G}_1 A$, and use L_d once more to obtain A. This proves H_d. □

The dependency order is explicit:

$$L_0,\ H_0,\ L_1,\ H_1,\ L_2,\ H_2, \ldots.$$

Conversion at depth d uses assumption access only at depth $d-1$. The proof is an external induction on finite formula syntax. It neither adds quantified induction to arithmetic nor presupposes the hypothesis access that it is proving.

Why bound coordinates matter. One cannot replace this argument by a casual appeal to one global equality substitution. The leaf terms may depend on coordinates bound inside the D prefix, so the endpoint equations live underneath those binders. The depth argument lifts those pointwise equations correctly.

5 Exact translation from the established clause calculus

The source calculus C is the raw Eq/D clause calculus formalized as `CPrf` in `NativeClausal.lean` [2]. It has R, the same two Eval cases, Use, Open, and the same witness-selected Intro. Its context entries are pairs $[A; y.B]$, read $A \to \forall y B$. Source Use requires a derivation of the stored antecedent and returns a chosen instance of the stored body.

Consider a source derivation beginning with ordinary assumptions $\mathrm{Asm}(A_i) = [0 = 0; v.A_i]$. Mark every context *occurrence* by origin:

origin	source clause	target body
ordinary assumption	$[0 = 0; v.A]$	$\mathsf{G}_v A$
opening a paired formula	$[B(b,0); y.B(b,y)]$	$B(b,y)$.

Different occurrences can have different origins even when their source clauses happen to be identical. These annotations belong to the proof translation only; the new checker stores neither origin tags nor pairs.

Theorem 5.1 (Constructive completeness translation)**.** *For Eq/D formulas $A_1, \ldots, A_n, F$,*

$$\mathrm{Asm}(A_1), \ldots, \mathrm{Asm}(A_n) \vdash_{\mathsf{C}} F \quad \Longrightarrow \quad \mathsf{G}A_1, \ldots, \mathsf{G}A_n \vdash_{\mathsf{B}} F. \tag{C-to-B}$$

Proof. Induct on the source proof, carrying the occurrence annotations. R and Eval translate unchanged. If Use selects an ordinary-assumption occurrence, replace that step by $H_{\mathrm{depth}(A)}$; its supplied proof of $0 = 0$ need not be used. If Use selects an Open occurrence, its antecedent is exactly $B(b,0)$, so the translated step is Use.

Source Open appends precisely an entry of the second kind, which the target Open appends as its single body. Source Intro appends precisely an ordinary assumption $B(w,\eta)$, whose translation is exactly the gate body added by target Intro. Fresh numerical parameters and the mode test are unchanged. Translate both branch derivations recursively. □

This argument needs no target cut theorem and no unrestricted substitution into target proof trees. It also does not assert that every arbitrary two-field clause is literally one self-based clause. The invariant concerns the origins reachable in the proofs needed for all ordinary sequents, which is sufficient for the theorem.

6 Soundness and the bi-interpretation

6.1 The ordinary scalar intermediary

The established intermediary T_{eq} is intuitionistic first-order logic with equality over the same scalar terms. Its nonlogical basis consists of the two iterator computation equations, numerical coverage

$$\forall x\,(x = 0 \ \vee \ \exists y\, x = Sy),$$

separation $0 = S0 \to \bot$, and induction for literal equations with side assumptions. The pinned project proves its bi-interpretation with iPRA, including both internal round trips [5]. We use that result as an established upstream theorem, not as a new formalization supplied here.

Lemma 6.1 (Meaning of a gate context)**.** *In T_{eq}, for every Eq/D formula A,*

$$\mathcal{Q}_v(\mathsf{G}_v A) \iff A^\circ. \tag{Assumption}$$

Proof. The zero endpoint is a true reflexive D prefix, and every successor endpoint is equivalent to A°, by computation and the ordinary logical congruence of the reading of D. From the left side, supply its true zero antecedent and specialize at 1 to obtain A°. Conversely, from A°, numerical coverage proves the gate at every argument: at zero use its true zero endpoint; at a successor use A°. This gives the displayed closure. □

Theorem 6.2 (Soundness in the theory, not just the standard model)**.** *If $\Gamma \vdash_{\mathsf{B}} A$, then T_{eq} derives A° from the ordinary readings $\mathcal{Q}B$ of the bodies in Γ.*

Proof. Induct on the derivation. R is equality reflexivity; Eval follows from computation and equality. Use is implication elimination followed by universal specialization. Open is existential elimination for the ordinary reading of D, with the stated eigenparameter condition.

For Intro, the added body has reading equivalent to $B(w, \eta)^\circ$ by the preceding lemma. In case mode the translated premise proves $B(w, Sz)$ from $B(w, 0)$; coverage yields all instances, and existential introduction supplies witness w. In progress mode the matrix is a literal equation and the premise is exactly its progressive step. Literal-equation induction gives the conditional universal closure, again with witness $w = 0$. All passive assumptions remain in place. □

Combining soundness, the gate equivalence, the upstream native/T_{eq} completeness and native formula round-trip theorems, the source proof translation, and `cprf_iff_nprf_of_qfree` gives the precise main result. The native/T_{eq} bridge is needed in the direction from ordinary soundness back to native derivability.

Theorem 6.3 (Exact ordinary-sequent equivalence)**.** *For Eq/D formulas $A_1, \ldots, A_n, F$,*

$$\mathsf{G}A_1, \ldots, \mathsf{G}A_n \vdash_{\mathsf{B}} F \iff A_1, \ldots, A_n \vdash_{\mathsf{N}} F. \tag{Equivalence}$$

Here the right side is the pinned native scalar calculus, on its Q-free fragment.

6.2 Both directions on terms and formulas

The equivalence just proved is the identity on raw terms and object Eq/D formulas. Only the representation of assumptions changes, with its internal equivalence already proved. Eliminate a native $Q_y B$ by $D_{x,y} B$ with x fresh and unused; the numerical domain is inhabited by 0. This is the project's `unQ` translation [2].

For clarity, the inherited numerical maps have the following form. In the scalar-to-iPRA direction, each fixed scalar iterator is compiled to a primitive recursive definition with its seed and passive variables as parameters. In the reverse direction, zero, successor and projections are immediate, and composition is substitution. A primitive recursion

$$f(0, \vec{a}) = g(\vec{a}), \quad f(Sn, \vec{a}) = h(n, f(n, \vec{a}), \vec{a})$$

is compiled with the already-derived scalar pairing and projections as

$$f(n,\vec{a}) = \pi_2 I\big(n, \langle 0, g(\vec{a})\rangle; u.\langle S\pi_1 u, h(\pi_1 u, \pi_2 u, \vec{a})\rangle\big).$$

Here g and h on the scalar right side denote their already-compiled scalar translations. The pairing, predecessor, reflection of equality, and correctness of this compiler are inherited from the upstream scalar bridge. We do not assume them in proving the new context theorem.

Let τ be the established scalar-to-PR term map and σ its inverse interpretation. Define the interpretation from the new calculus by the ordinary reading followed by τ. Define the reverse map by the upstream logical-macro translation after σ, then `unQ`. The upstream theorems `native_roundTrip` and `pra_native_roundTrip`, transported through (Equivalence), prove that every formula is internally equivalent to its round trip.

Both numerical domains are whole; equality is actual equality. The composite numerical interpretations have graph $x = y$, as in the upstream `BiInterp` theorem. No restriction to standard numbers, quotient domain, external truth test, or merely equiconsistent surrogate is introduced by this construction.

Corollary 6.4. *The new calculus is bi-interpretable with full first-order iPRA and*

$$\varnothing \vdash_{\mathsf{B}} 0 = S0 \quad \Longleftrightarrow \quad \text{iPRA} \vdash \bot.$$

7 The entire ordinary logical interface

This section makes the interpretation of arbitrary ordinary formulas explicit. The macros are derived operations, not new rules of the checker. Their validity follows from the established native logical interface and (Equivalence) [4].

First, $\mathcal{Q}_y B$ can be written in the object language as $D_{x,y}B$ with unused x. Define a depth-raising envelope

$$L(A) := D_{x,y}\mathsf{G}_y A, \qquad x \notin \mathrm{FV}(A).$$

Its ordinary reading is equivalent to A by (Assumption). Repeated L-padding can therefore align the D depths of any two formulas. After aligning and alpha-renaming their prefixes, define

$$\mathrm{Sel}_v(A,B) = D^d\big(\mathsf{C}(v, s_A, s_B) = \mathsf{C}(v, t_A, t_B)\big).$$

It has endpoint A at zero and endpoint B at every successor. The notation D^d here abbreviates the common, explicitly bound prefix.

Ordinary connective	Native macro
$\bot$	$0 = S0$
$A \to B$	$\mathcal{Q}_y\,\mathrm{Sel}_y(A,B)$
$A \wedge B$	$\mathcal{Q}_y \mathsf{G}_y\,\mathrm{Sel}_{\mathrm{par}(y)}(A,B)$
$\exists x\, A(x)$	$D_{x,y}\mathsf{G}_y A(x)$
$A \vee B$	$\exists v\ \mathrm{Sel}_v(A,B)$
$\forall x\, A(x)$	$A(0) \wedge \mathcal{Q}_y A(y)$

Freshness is understood in every macro. The displayed existential, conjunction and universal symbols on right sides are expanded using the earlier rows. The parity term can be taken as

$$\mathrm{par}(n) = I(n, 0; u.\mathsf{C}(u, 1, 0)).$$

Its needed arithmetic properties are derived in the native theory. Implication works because its zero selector instance is A, and its successor instances are B. The conjunction gate has a true zero base; its values at 1 and 2 supply the two conjuncts. Existential quantification works because the gated base is true, making the conditional closure equivalent to the matrix at the selected witness.

A padding trap. Plain vacuous D padding would be incorrect. If both coordinates are unused, $D_{x,y}A$ reads $\exists x(A \to A)$ and is always true. The envelope $L(A)$ above is essential. This distinction matters when combining formulas of different depths.

8 Checker implementation and evidence

8.1 Public contract

The new reference checker accepts a JSON object with exactly the keys `goal` and `proof`. The goal must be a closed well-formed Eq/D formula. Successful verification prints `ACCEPT`; parsing, typing, scoping, shape or proof mismatches reject. Like the matched control, its practical execution is subject to host memory and recursion limits; the mathematical checker is the corresponding finite recursion without a fixed resource cap.

The codec uses 0 for zero, negative integers for fresh free-parameter levels, positive integers for bound indices, `[0,t]` for successor, and `[n,a,h]` for iteration. Equation and paired formulas are `[s,t]` and `[B]`. In a D matrix, y has index 1 and x index 2. Exact integer tests exclude JSON booleans.

The five proof packet tags are `r`, `v`, `u`, `e`, and `i`. Their detailed data fields and runnable examples are described in the accompanying README. All contexts are constructed by the verifier; the input cannot inject an arbitrary retained body. Both equality premises of Eval must evaluate to equations. Fresh levels are chosen from the scope counter and cannot escape an Open or Intro branch into its outer conclusion.

For a formula of depth d, the gate's external clause coordinate is placed at bound index $2d+1$. Each old endpoint is lifted once under the new iterator update binder. This is the implementation point most likely to be wrong in an informal "wrap the endpoints" translation. The package includes an independent numerical-environment test of these scope operations.

8.2 Matched costs and separate refinements

All principal sizes are computed from `ast.unparse(ast.parse(source))`, followed by one LF byte, on CPython 3.12.14. Significant tokens exclude whitespace and structural tokenizer markers; AST nodes count all nodes yielded by `ast.walk`. The executable measurement script and exact sources are included.

Complete checker	Tokens	AST	Bytes
Matched raw two-field control	848	986	2,999
New raw one-body calculus, atomic R	901	1,041	3,130
Existing full Eq/D compact checker	358	413	788
Same existing calculus, preferred refinement	357	412	778
Same existing calculus, byte-priority variant	357	415	777

The mathematical comparison adds about 4.37% in normalized bytes. The cost is the explicit gate construction and capture-avoiding lifting; deleting the redundant context field does not pay for that machinery in this implementation. A tested variant with primitive deep reflexivity was larger still, at 3,208 bytes. Deep reflexivity is therefore kept as a derived lemma in the main calculus.

The preferred 778-byte checker saves 10 bytes (about 1.27%), one significant token and one AST node relative to the existing full Eq/D checker. Its changes are equivalent validation expressions; its calculus is unchanged. The byte-priority variant additionally rewrites a Boolean dispatch and saves 11 bytes in total, but adds two AST nodes relative to the baseline. It therefore does not improve all three coordinates. The guarded compact 516-byte codec reported by `canonical_recursors` has a different public validation contract. Its size must not be used as though it were the full Eq/D checker. The reported 256-token and 278-AST minima also belong to other programs; combining coordinatewise minima into one imaginary checker would be misleading.

8.3 What was checked

The package contains executable positive certificates, deliberately invalid certificates, normalized source measurements, the independent gate-scope test, and a source-level equivalence audit of the compact implementation refinement. The final machine-readable evidence records the exact checks and their counts. Reproduction commands are in the README; scripts write their own evidence rather than relying on an unverifiable narrative of a run. The executed suite passed 140 standalone process checks: 34 raw-calculus certificates in both ordinary Python and `python -O`, and 12 inherited native fixtures for each of three checkers in both modes. It also passed 18,882 validator cases, producing 37,764 candidate/control comparisons for the two refined validators.

The positive certificates include genuine equation induction using its gated hypothesis, ordinary Open/Use, both Eval cases, derived joining, and nested-D assumption extraction. In the last example the extracted assumption is false in the standard model, so its branch cannot succeed by ignoring that assumption. Invalid certificates exercise malformed types, shape mismatches, indices, and capture attempts.

The focused independent gate test checks 36,462 cases, including D depths zero through three, nested iterator binders, bound logical coordinates, free parameters, and opening at numerical arguments zero, one and two. Its evaluator uses explicit environments and does not implement its expected result by calling the gate transformation again.

These tests are supplementary evidence. The new soundness, completeness and bi-interpretation transfer are written metatheorems, independently reviewed during this work. They have not been compiled as new Lean or Rocq proofs. The inspected upstream theorem files are included for provenance, but the upstream Lean project was not rebuilt in this run. No standard-model test is presented as a proof of bi-interpretability.

9 Further research questions

1. **Can one-body contexts reduce a complete checker?** The present theorem removes an independent field, but explicit gates cost more than that field. A useful next step must amortize gate lifting or find an equally complete assumption representation with cheaper scope operations. Any claim should include the whole validator.

2. **Can hypothesis access be built into a smaller uniform inference without a hidden tag?** The source-origin proof shows that only assumptions and self-based closures are needed. Storing an extra origin tag is a simpler implementation idea, but its measured direct variants did not improve the control. A new inference discipline that avoids both tags and gates would be more substantial.

3. **Can the new metatheorem be formalized with a small dependency footprint?** A Lean implementation should first formalize the pointwise gate and its binding identities, then atomic equality, derived deep reflexivity, and the simultaneous L_d/H_d argument. Occurrence-origin annotations can give completeness without first proving target cut.

4. **Can the introduction be made easier to substitute?** The witness-selected mode is retained here. The repository already has diagonal alternatives. Their combination with one-body contexts may improve proof transformations, but would need a fresh matched cost comparison and should not be counted as a new diagonal construction.

5. **What is a useful lower-bound question?** There is no global minimality proof here. Restricted classes of presentations—fixed term syntax, explicit primitive-recursive scope operations, complete input validation and a stated language/runtime—may support meaningful lower bounds or exhaustive comparisons. A byte count alone, with an unspecified trusted interpreter, cannot establish an absolute simplest axiomatization.

Reproducibility and attribution

All claims about the repository refer to the pinned commit, not to a moving branch. The term language, D reading, witness-selected mode, Eval subsystem, source clause calculus and scalar/iPRA bridge are inherited. The contribution proved here is their one-body context presentation and the exact occurrence-origin translation.

The deliverable contains this source and compiled PDF, the complete new checker, a matched control, the separately refined compact checker and its baseline, verification scripts and certificates, source provenance, and a checksum manifest. Reference source files remain unmodified and are not advertised as a self-contained Lean project.

References

[1] V. Reshetnikov, ProveIt, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at the pinned commit. Repository README.

[2] ProveIt, `NativeClausal.lean`: `CPrf`, `cprf_iff_nprf_of_qfree` and `cprf_iff_nprf`. Pinned source.

[3] ProveIt, `NativeEval.lean`: two-case computational equality and `eprf_iff_nprf`. Pinned source.

[4] ProveIt, `NativeLogic.lean`, `NativeComposite.lean`: the ordinary logical interface and its native composites. Pinned logical rules.

[5] ProveIt, `BiInterpretation.lean`: the scalar/iPRA `BiInterp`, theorem translations and round trips. Pinned source.

[6] ProveIt, `NativeBiInterpretation.lean`: `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`. Pinned source.